骑步行导航接口文档

目录

- **Android 步骑行导航接口文档**
  - 目录
  - 对比7.+版本接口修改：
        - a.创建多实例
        - b.设置地图车标偏移
  - 1. 新增接口
    - 1.1 多实例地图 DPI 缩放
    - 1.2 骑行导航全览路线 Margin 设置
    - 1.3 车标 Heading 调节
    - 1.4 行中动态设置地图元素
      - 主实例
      - 多实例
    - 1.5 行中定位回调
    - 1.6 导航设置管理器
    - 1.7 TTS 播报接口更新
    - 1.8 导航中元素显隐控制
      - 1.8.1 多实例地图
      - 1.8.2 主地图
    - 1.9 多实例截图接口更新
    - 1.10 个性化地图设置
    - 1.11 自定义导航样式
    - 1.12 模拟导航
      - 骑行
      - 步行
    - 1.13 多实例地图 POI 显示控制
    - 1.14 红绿灯数据透出
    - 1.15 导航状态常驻
    - 1.16 TTS 播报开关
    - 1.17 车道级模式设置
    - 1.18 鹰眼小图控件
    - 1.19 隐藏骑行导航中自带布局
  - 2. 接口详细说明
    - 2.1 多实例相关接口
  - 3. Q&A
    - 3.1 如何设置车辆图标在地图中的位置?
    - 3.2 如何设置导航时缩放级别?
    - 3.3 副地图显示地图 Logo?
    - 3.4 如何隐藏地图背景只保留路线以及道路信息?
    - 3.5 设置了步骑行动态红绿灯不渲染咋回事?
    - 3.6 动态红绿灯为什么不展示?
    - 3.7 如何创建多个多实例地图?
  - 4. License 鉴权
  - 5. 更新日志
    - 2026-05-11
    - 2025-03-11
    - Bugfix

Android 步骑行导航接口文档

> 文档版本: v1.1
> 最后更新: 2026-05-26
> 适用平台: Android
> 模块: 步骑行导航 SDK

目录

1. 新增/更改接口
2. 接口详细说明
3. Q&A
4. License 鉴权
5. 更新日志

对比7.+版本接口修改：

a.创建多实例

参考 3.7如何创建多个多实例地图?

b.设置地图车标偏移

单位的修改，从像素改为比例缩放。

参考 3.1如何设置车辆图标在地图中的位置

1. 新增接口

1.1 多实例地图 DPI 缩放

接口签名:

```java
void setMapDpiScale(float mapScale, float logoScale)
```

说明: 接口增加第二个参数，表示设置多实例 logo 缩放比例。设置成 0 不显示 logo。

参数:

参数名	类型	说明
mapScale	float	地图缩放比例
logoScale	float	Logo 缩放比例，设为 0 不显示 logo

1.2 骑行导航全览路线 Margin 设置

说明: 骑行导航增加设置全览路线的 margin 接口。

注意事项: 设置导航中全览 margin 要保证 `NaviHelper.getNaviMap()` 已经添加可以计算出来宽高。

示例代码:

```java
mNaviHelper.getNaviMap().post(new Runnable() {
    @Override
    public void run() {
        // 四个参数分别代表，距离 NaviMapView 左上右下像素值
        mNaviHelper.setRouteMargin(500, 0, 0, 0);
    }
});
```

1.3 车标 Heading 调节

接口签名:

```java
void setNeedSensor(boolean needSensor)
```

说明: `BikeNavigateHelper` 增加 `setNeedSensor(bool)`，用来调节车标 heading。不设置 sensor（设置为 false）车头默认会按照路线方向。

调用时机: 需要在开始导航之前调用。

1.4 行中动态设置地图元素

主实例

```java
BikeNaviDisplayOption.BikeMapCustomDrawOption bikeNaviDisplayOption =
    new BikeNaviDisplayOption.BikeMapCustomDrawOption();

bikeNaviDisplayOption.setCompassCustomBitmap();      // 设置罗盘资源
bikeNaviDisplayOption.setEndPointCustomBitmap();    // 设置终点资源
bikeNaviDisplayOption.setCarPointCustomRes();       // 设置车标资源
bikeNaviDisplayOption.setRouteNormalCustomBitmap(); // 设置未走过路线资源
bikeNaviDisplayOption.setRoutePassedCustomBitmap(); // 设置已走过路线资源
bikeNaviDisplayOption.setRouteCustomWidth();        // 设置路线宽度
bikeNaviDisplayOption.setCarPointCustomScale();     // 设置车标缩放比例
bikeNaviDisplayOption.setEndPointCustomScale();     // 设置终点缩放比例
bikeNaviDisplayOption.setCompassCustomScale();      // 设置罗盘缩放比例

BikeNavigateHelper.getInstance().updateMapRenderCustomDrawOption(bikeNaviDisplayOption);
```

多实例

```java
MapCustomDrawOption mapCustomDrawOption = new MapCustomDrawOption();

mapCustomDrawOption.setCompassCustomBitmap();      // 设置罗盘资源
mapCustomDrawOption.setEndPointCustomBitmap();    // 设置终点资源
mapCustomDrawOption.setCarPointCustomRes();       // 设置车标资源
mapCustomDrawOption.setRouteNormalCustomBitmap(); // 设置未走过路线资源
mapCustomDrawOption.setRoutePassedCustomBitmap(); // 设置已走过路线资源
mapCustomDrawOption.setRouteCustomWidth();        // 设置路线宽度
mapCustomDrawOption.setCarPointCustomScale();     // 设置车标缩放比例
mapCustomDrawOption.setEndPointCustomScale();     // 设置终点缩放比例
mapCustomDrawOption.setCompassCustomScale();      // 设置罗盘缩放比例

multiNaviView.updateExtMapRenderCustomDrawOption(mapCustomDrawOption);
```

1.5 行中定位回调

接口签名:

```java
void onGpsStatusChange(int gpsSignalLevel)
```

说明: GPS 状态发生变化，来自诱导引擎的消息

参数:

参数名	类型	说明
gpsSignalLevel	int	GPS 信号强弱等级：0-弱，1-强（暂时只有两个状态）

回调接口: `IWRouteGuidanceListener`

1.6 导航设置管理器

接口签名:

```java
NaviSettingManager getNaviSettingManager()
```

说明: `BikeNavigateHelper` 增加接口 `getNaviSettingManager()`，用于设置起终点红线显隐等。

可用方法:

方法	说明
`setVibrationOpen()`	震动提醒开关
`setRedlineOpen()`	起终点红线显隐开关
`setCrossMapLevelOpen()`	路口自动放大开关

1.7 TTS 播报接口更新

接口签名:

```java
int playTTSText(String speech, int bPreempt, int type)
```

说明:

- 第二个参数 `int bPreempt` 用来判断当前的 tts 播报是否需要抢占焦点
- `getTTSState` 接口已废弃

参数:

参数名	类型	说明
speech	String	播报文本
bPreempt	int	抢占焦点标识：1-优先级较高建议抢占焦点，0-优先级较低建议播报完成上条 tts
type	int	播报类型

示例代码:

```java
WNTTSManager.getInstance().initTTS(new WNTTSManager.IWNOuterTTSPlayerCallback() {
    @Override
    public int playTTSText(String speech, int bPreempt, int type) {
        // bPreempt = 1 表示优先级较高 建议抢占焦点
        // bPreempt = 0 表示优先级较低，建议播报完成上条 tts
        return 0;
    }

    @Override
    @Deprecated
    public int getTTSState() {
        return 0;
    }
});
```

1.8 导航中元素显隐控制

元素类型枚举:

常量	值	说明
`CAR_TO_END_RED_LINE`	`1`	到终点红线
`TRAFFIC_LIGHT`	`1 << 1`	红绿灯标签
`TRAFFIC_LIGHT_POP`	`1 << 2`	红绿灯泡泡
`ROAD_NAME_POP`	`1 << 3`	路名泡泡
`MILESTONE`	`1 << 4`	里程碑标签
`FACILITY`	`1 << 5`	交通设施标签
`FACILITY_POP`	`1 << 6`	交通设施泡泡

1.8.1 多实例地图

```java
/**
 * 控制元素的显隐
 * @param isShow 控制元素的显隐
 * @param elements 要控制的元素列表
 */
void setNaviDrawElementsShow(boolean isShow, List<Element> elements)
```

1.8.2 主地图

```java
/**
 * 控制元素的显隐
 * @param isShow 控制元素的显隐
 * @param elements 要控制的元素列表
 */
BikeNavigateHelper.getInstance().setNaviDrawElementsShow(boolean isShow, List<Element> elements)
```

1.9 多实例截图接口更新

接口签名:

```java
void snapshotScope(SnapshotReadyCallback snapshotReadyCallback, boolean needLogo)
```

说明: 多实例截图接口增加第二个参数，bool 类型，截屏是否带上地图 logo。

1.10 个性化地图设置

接口签名:

```java
void setMapCustomStylePath(String stylePath)
```

说明: 多实例支持设置个性化地图，传入 sty 文件的路径。

示例代码:

```java
// multi_style_test.sty 文件放置在 sdcard/Android/data/包名/files/ 下面
multiNaviView.setMapCustomStylePath(getExternalFilesDir(null) + "/multi_style_test.sty");
```

1.11 自定义导航样式

说明: 如何自定义导航路线宽度、颜色? 如何自定义起止点及车辆图标?

示例代码:

```java
EngineOptions options = new EngineOptions.Builder()
    .setRouteCustomWidth(30)                    // 路线宽度最大 30
    .setRouteNormalCustomBitmap()               // 未走过路线纹理
    .setRouteNormalCustomBitmapExt()            // 多实例未走过路线纹理
    .setRoutePassedCustomBitmap()               // 已走过路线纹理
    .setRoutePassedCustomBitmapExt()            // 多实例已走过路线纹理
    .setCarPointCustomRes()                     // 自车位置
    .setCarPointCustomResExt()                  // 多实例自车位置
    .setEndPointCustomBitmap()                  // 终点图片
    .setEndPointCustomBitmapExt()               // 多实例终点图片
    .setExtUseMainRes()                         // 多实例地图是否使用主地图的资源，默认使用默认资源
    .build();

// 骑行导航
BikeNavigateHelper.getInstance().initNaviEngine(this, options, new IBEngineInitListener());

// 步行导航
WalkNavigateHelper.getInstance().initNaviEngine(this, options, new IBEngineInitListener());
```

1.12 模拟导航

骑行

```java
// 开始模拟导航
BikeNavigateHelper.getInstance().startBikeNavi(this, BikeNavigateHelper.NaviMode.FakeNavi);

// 设置模拟导航速度 m/s
BikeNavigateHelper.getInstance().setSimulateNaviSpeed(5);
```

步行

> 注意: AR 导航不支持模拟导航

```java
// 开始模拟导航
WalkNavigateHelper.getInstance().startBikeNavi(this, WalkNavigateHelper.NaviMode.FakeNavi);

// 设置模拟导航速度 m/s
WalkNavigateHelper.getInstance().setSimulateNaviSpeed(5);
```

1.13 多实例地图 POI 显示控制

接口签名:

```java
/**
 * 是否显示多实例地图的 poi
 * @param isShow true 显示 默认显示
 */
void showPoiMark(boolean isShow)
```

1.14 红绿灯数据透出

前置条件: 需要有红绿灯高级权限。

调用方法:

```java
helper.setShowLight(true);
```

回调接口: `IBRouteGuidanceListener`

```java
/**
 * 红绿灯数据透出
 */
void onTrafficLightOutDataUpdate(TrafficLightOutData trafficLightOutData)
```

设置监听:

```java
public void setRouteGuidanceListener(Activity activity, IBRouteGuidanceListener routeGuidanceListener) {
    if (null != routeGuidanceListener) {
        bikeNaviManager.setRouteGuidanceListener(activity, routeGuidanceListener);
    }
}
```

数据模型:

```java
public class TrafficLightOutData {
    /**
     * renderType: 渲染类型
     * 0-刷新红绿灯气泡（开始展示气泡或者灯态数据过期需要更新）
     * 1-隐藏红绿灯气泡
     * 2-隐藏红绿灯 icon
     */
    private int renderType;

    /**
     * direction: 方向
     * 0-不带方向的灯
     * 1-直行灯
     * 2-左转灯
     */
    private int direction;

    /**
     * popType: 气泡类型
     * 0-无效值
     * 1-普通态
     * 2-大灯态
     * 3-小灯态
     */
    private int popType;

    /**
     * lightType: 灯类型
     * 0-无效值
     * 1-直行或左转单灯
     * 2-左转双灯第一个灯
     * 3-左转双灯第二个灯
     */
    private int lightType;

    public static class TrafficLightDataInfo {
        /**
         * status: 灯状态
         * 0-无效值
         * 11-灭灯
         * 21-红灯
         * 22-黄灯（黄灯的气泡不显示倒计时，用文案"注意"代替）
         * 23-绿灯
         */
        private int status;

        /**
         * period: 周期
         * 使用时，每过一秒，period 减一。
         * 当 period <= 0 时，取红绿灯态数组的下一份数据继续展示使用
         */
        private int period;

        /**
         * countDown: 倒计时
         * 只在 0 < countDown <= 999 的情况下展示倒计时。
         * 如果返回的倒计时为 10000 或者 countDown 走完归零 period > 0 的情况下，
         * 红绿灯气泡改为显示文案。
         * 具体为红灯显示"等待"，黄灯显示"注意"，绿灯显示"通行"。
         */
        private int countDown;
    }
}
```

1.15 导航状态常驻

接口说明:

接口	说明
`setIfNaviStanding(true)`	设置导航状态常驻，退出页面导航正常进行
`isNaviStanding()`	获取是否常驻导航
`isNavigating()`	获取导航状态，导航中返回 true

开发者注意事项:

原有的导航页面的 destroy 方法改动：

```java
@Override
protected void onDestroy() {
    super.onDestroy();
    if (!mNaviHelper.isNaviStanding()) {
        mNaviHelper.quit();  // 正式退出导航，释放引擎
    } else {
        mNaviHelper.onDestroy(false);
    }
}
```

> 注意: `BikeNavigateHelper.quit()` 方法正式退出导航，释放引擎。之后需要重新初始化可以参考 DEMO 实现。

1.16 TTS 播报开关

接口签名:

```java
void enableTTS()
```

说明: 新增 TTS 播报开关，开启/关闭 tts 播报，默认开启。

调用方式:

```java
WNTTSManager.getInstance().enableTTS();
```

1.17 车道级模式设置

说明:

- 车道级仅支持自行车、电动车
- 车道级与 2d/3d 全览模式切换互斥

接口:

类型	接口
主实例	`BikeNavigateHelper.enterLaneNaviMode()`
多实例	`IMultiNaviView.enterLaneNaviMode()`

1.18 鹰眼小图控件

说明: 行中可以添加小地图控件。`EagleMiniMap` 是个 View 可以 xml 引入到导航布局里面。

1.19 隐藏骑行导航中自带布局

a. BikeNaviDisplayOption中增加loadNaviOverlayLayout方法

b.BikeNavigateHelper 增加 setLoadNaviOverlayLayout 方法 传入 false可隐藏

---

2. 接口详细说明

2.1 多实例相关接口

参考 3.7 如何创建多个多实例地图。

获取到IMultiNaviViewProxy类，可设置多实例功能。

```java
 /**
     * lifecycle callback
     */
    void onResume();
    /**
     * lifecycle callback
     */
    void onPause();
    /**
     * lifecycle callback
     */
    void onDestroy();

    /**
     * 设置导航类型 步行、骑行
     * 创建之后设置，不设置多实例不生效
     * @param type 导航类型
     * @param tag 标签
     * @return 是否设置成功
     */
    boolean setNaviType(int type, int tag);

    /**
     * 设置地图旋转模式 车头朝向模式、地图北朝向模式
     * @param model 地图旋转模式
     * @return 是否设置成功
     */
    boolean setRotateMode(Map_Rotate_Mode model);

    /**
     * 3d 2d 模式设置
     * @param mode 3d 2d 模式
     * @return 是否设置成功
     */
    boolean setDefaultOverlooking(OverLookingMode mode);

    /**
     * 设置地图默认缩放级别
     * @param level 缩放级别
     * @return 是否设置成功
     */
    boolean setDefaultLevel(float level);

    /**
     * 暂时未用到
     * @param enable
     * @return
     */
    boolean setSupBackgroundDraw(boolean enable);

    /**
     * 暂时未用到
     * @return
     */
    boolean isSetBackgroundDraw();

    /**
     * 设置地图暂停也去绘制
     * @param enable
     */
    void setMapPausedDraw(boolean enable);

    /**
     * 获取地图视图
     * @return 地图视图
     */
    MapTextureView getMapTextureView();

    /**
     * 截取地图屏幕
     * @param callback 回调
     * @param needLogo 是否需要logo
     */
    void screenshot(SnapshotReadyCallback callback, final boolean needLogo);

    /**
     * 设置多实例地图dpi基于主图的缩放系数
     *
     * @param mapScale 地图元素缩放系数
     * @param logoScale logo缩放系数
     */
    void setMapDpiScale(float mapScale, float logoScale);

    /**
     * 设置多实例个性化地图
     *
     * @param customStyleFilePath style文件地址
     * @return 是否设置成功
     */
    boolean setMapCustomStylePath(String customStyleFilePath);

    /**
     * 设置多实例地图dpi基于主图的缩放系数
     *
     * @param scale 缩放系数
     */
    void setMapDpiScale(float scale);


    /**
     * 地图截屏回调接口
     */
    interface SnapshotReadyCallback {
        /**
         * 地图截屏回调接口
         *
         * @param snapshot 截屏返回的 bitmap 数据
         */
        void onSnapshotReady(Bitmap snapshot);
    }

    /**
     * 朝向模式
     */
    enum Map_Rotate_Mode {
        /**
         * 路线朝上
         */
        Map_Rotate_Route,
        /**
         * 正北朝上
         */
        Map_Rotate_North
    }

    /**
     * 设置padding
     *
     * @param l left
     * @param t top
     * @param r right
     * @param b bottom
     */
    void setPadding(int l, int t, int r, int b);
    /**
     * 设置Logo位置（不设置默认左下角）
     *
     * @param position 枚举类LogoPosition
     */
    void setLogoPosition(LogoPosition position);

    /**
     * 多实例导航元素绘制控制接口
     *
     * @param isShow 控制元素的显隐
     * @param elements 要控制的元素列表
     * @return
     */
    boolean setNaviDrawElementsShow(boolean isShow, List<NaviDrawElementType> elements);

    /**
     * 更新外部自定义渲染配置
     * @param option
     * @return
     */
    int updateExtMapRenderCustomDrawOption(MapCustomDrawOption option);

    /**
     * 显示隐藏3d楼块
     * @param enabled
     */
    void setBuildingsEnabled(boolean enabled);

    /**
     * 设置多实例视图创建回调
     * @param multiViewCreateCallback
     */
    void setMultiViewCreateCallback(OnMultiViewCreateCallback multiViewCreateCallback);

    /**
     * 多实例视图创建回调接口
     */
    interface OnMultiViewCreateCallback {
        void onBind();
        void onUnbind();
    }

    /**
     * 是否显示多实例地图的poi
     * @param isShow true 显示 默认显示
     */
    void showPoiMark(boolean isShow);

    /**
     * 设置多实例地图全览边距 MapMargin
     * 该接口设置要在开启全览{@link #setNaviMapViewAllStatus(boolean isNaviMapViewAllStatus)} 之前设置
     * @param left 距离地图左边界像素值
     * @param top 距离地图上边界像素值
     * @param right 距离地图右边界像素值
     * @param bottom 距离地图下边界像素值
     */
    void setNaviMapMargin(int left, int top, int  right, int bottom);


    /**
     * 设置多实例地图是否开启全览
     * 注：需要进入导航后多实例路线加载完成之后设置。
     *
     * @param isNaviMapViewAllStatus 设置是否需要开启全览，true 为开启，false 关闭全览
     */
    void setNaviMapViewAllStatus(boolean isNaviMapViewAllStatus);

    /**
     * 设置多实例导航跟随状态或者地图浏览状态\n
     * 当系统处于地图浏览状态(例如正在浏览地图)时，导航引擎就不会通过出接口向外通知做相应刷新，\n
     * 当系统处于导航跟随状态(例如点击了回导航)时，导航引擎会通过出接口向外通知做相应刷新
     * @param bIsBrowseStatus true:地图浏览状态 false:导航跟随状态
     */
    void setExtBrowseStatus(boolean bIsBrowseStatus);

    /**
     * 设置骑行车道级模式
     * @param isLaneMode
     */
    void enterLaneNaviMode(boolean isLaneMode);
```

---

3. Q&A

3.1 如何设置车辆图标在地图中的位置?

注意: 需要在开始导航后调用。调用后不会马上更新位置，收到 gps 位置信息之后才会更新，可以用模拟导航验证。

骑行:

```java
/**
 * 行中设置自车位置偏移
 * @param x    X 轴偏移量 比例 0.3f  代表右方
 * @param y    Y 轴偏移量 比例 0.3f  代表下方
 * @param type 0 主图，1 多实例
 * @param tag  多实例 tag（多实例传对应 tag 值，主图传 0 即可）
 * @return boolean
 */
BikeNavigateHelper.getInstance().setCarPosOffset(x, y, type, tag);
```

步行:

```java
WalkNavigateHelper.getInstance().setCarPosOffset(x, y, type, tag);
```

3.2 如何设置导航时缩放级别?

骑行:

```java
// level = [4,22]
BikeNavigateHelper.getInstance().setDefaultNaviMapScale(level);
```

步行:

```java
// level = [4,22]
WalkNavigateHelper.getInstance().setDefaultNaviMapScale(level);
```

3.3 副地图显示地图 Logo?

```java
MultiNaviViewProvider.IMultiNaviViewProxy multiNaviView =
    MultiNaviViewProvider.getInstance().createDefaultMultiNaviView(BNaviMainActivity.this);

multiNaviView.setPadding(10, 10, 10, 10);

// 设置 logo 位置，不设置默认左下角
multiNaviView.setLogoPosition(LogoPosition.logoPostionleftBottom);

// Logo 位置枚举
enum LogoPosition {
    logoPostionleftBottom,    // 屏幕左下位置
    logoPostionleftTop,       // 屏幕左上位置
    logoPostionCenterBottom,  // 屏幕中下位置
    logoPostionCenterTop,     // 屏幕中上位置
    logoPostionRightBottom,   // 屏幕右下位置
    logoPostionRightTop       // 屏幕右上位置
}
```

3.4 如何隐藏地图背景只保留路线以及道路信息?

可以将这三个接口结合使用，参考 demo：

```java
// 设置个性化样式
multiNaviView.setMapCustomStylePath(getExternalFilesDir(null).getAbsolutePath() + "/map123.sty");

// 隐藏 POI
multiNaviView.showPoiMark(false);

// 设置路况显示
multiNaviView.getMapTextureView().setTraffic(true);
```

3.5 设置了步骑行动态红绿灯不渲染咋回事?

原因 1: 需要在算路之前设置。

原因 2: 锁屏后红绿灯不渲染? 可能调用了摩托车驾车 `BaiduNaviManagerFactory.getMapManager().onPause()` 接口。

原因 3: 还有设置打开后台渲染的。

排查步骤:

3.6 动态红绿灯为什么不展示?

检查项:

1. 确认是否有步骑行红绿灯高级权限
2. `helper.setShowLight(true)` 接口是否调用（需要在算路前调用）

3.7 如何创建多个多实例地图?

说明: 最多支持 3 个多实例地图。

示例代码:

```java
// 引擎初始化
BikeNavigateHelper.getInstance().initNaviEngine(this.getApplicationContext(), options,
    new IBEngineInitListener() {
        @Override
        public void engineInitSuccess() {
            BikeNavigateHelper.getInstance().setShowLight(true);
            Log.d(TAG, "BikeNavi engineInitSuccess multi");

            // 第 1 个多实例
            MultiNaviViewProvider.getInstance().createMultiNaviView(BNaviMainActivity.this);
            // 第 2 个多实例
            MultiNaviViewProvider.getInstance().createMultiNaviView(BNaviMainActivity.this);
            // 第 3 个多实例
            MultiNaviViewProvider.getInstance().createMultiNaviView(BNaviMainActivity.this);
            // 第 4 个多实例 超出不生效
            // MultiNaviViewProvider.getInstance().createMultiNaviView(BNaviMainActivity.this);

            // 多实例创建完毕，发起导航算路
            routePlanMultiWithBikeParam();
        }

        @Override
        public void engineInitFail() {
            Log.d(TAG, "BikeNavi engineInitFail");
            BikeNavigateHelper.getInstance().unInitNaviEngine();
        }
    });
```

获取多实例地图并添加到布局:

```java
multiNaviViewMap = MultiNaviViewProvider.getInstance().getAllMultiNaviView();
int index = 0;
for (Map.Entry<Integer, MultiNaviViewProvider.IMultiNaviViewProxy> entry : multiNaviViewMap.entrySet()) {
    Integer tag = entry.getKey(); // 多实例 tag
    MultiNaviViewProvider.IMultiNaviViewProxy multiNaviView = entry.getValue();

    // 设置导航类型和 tag
    multiNaviView.setNaviType(IMultiNaviView.TYPE_NAVI_RIDE, tag);

    FrameLayout.LayoutParams params = new FrameLayout.LayoutParams(400, 600);
    params.setMargins(1180, 200 + index * 600, 0, 0);

    // 注入多实例地图 view
    multiNaviView.injectMultiNaviView(frameLayout, params);
    index++;
}
```

---

4. License 鉴权

License 授权文档

---

5. 更新日志

2026-05-11

1.最多支持3个多实例地图。

2.导航车标增加平滑效果。

3.支持骑行车道级导航。

2025-03-11

Bugfix

1. 步骑行 SDK 自带导航中 UI 与多实例地图状态无联动问题
2. 当前速度偶现比最大速度还要大问题
3. 步骑行 SDK 自带导航中 UI 遮挡问题
4. 修复个性化底图闪白问题
5. 稳定性，修复收集到的 crash
6. 修复首次进入导航与设置的车标偏移冲突，导致车标跳动问题
7. 优化体验，设置了固定比例尺之后，首次进入地图会多次缩放的问题
8. `setRotateMode` 接口主地图调用时机优化，现在可以在引擎初始化完成之后任意时机调用。地图视角在设置了不同视角后，前后台切换还原问题修复
9. 设置了 carposoffset 之后 全览位置不对 偏移了
10. 关掉起点近大远小效果，防止闪烁
11. 多实例截图接口支持地图静止状态截图
12. `setOverlayUnderPoi` 设置路名名画在路线上兼容问题修复